

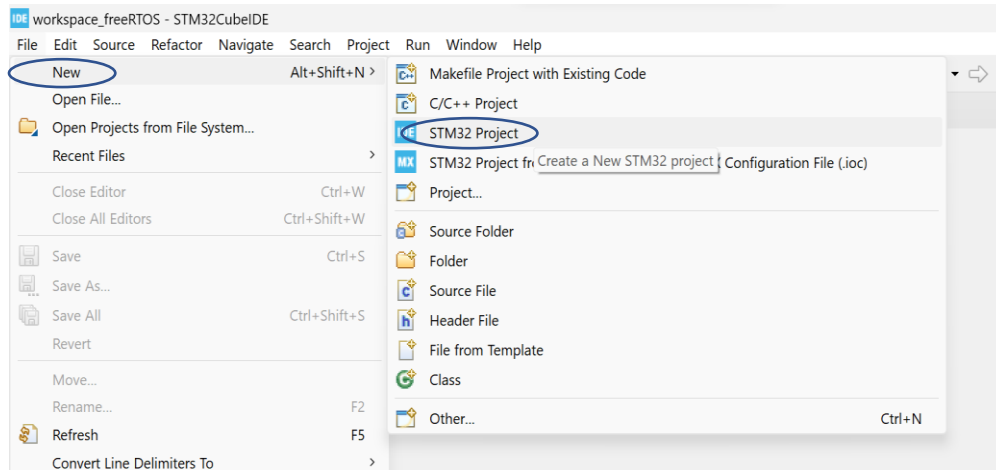


PRÁTICA 1 – Introdução ao STM32IDE e STM32F401RE NÚCLEO

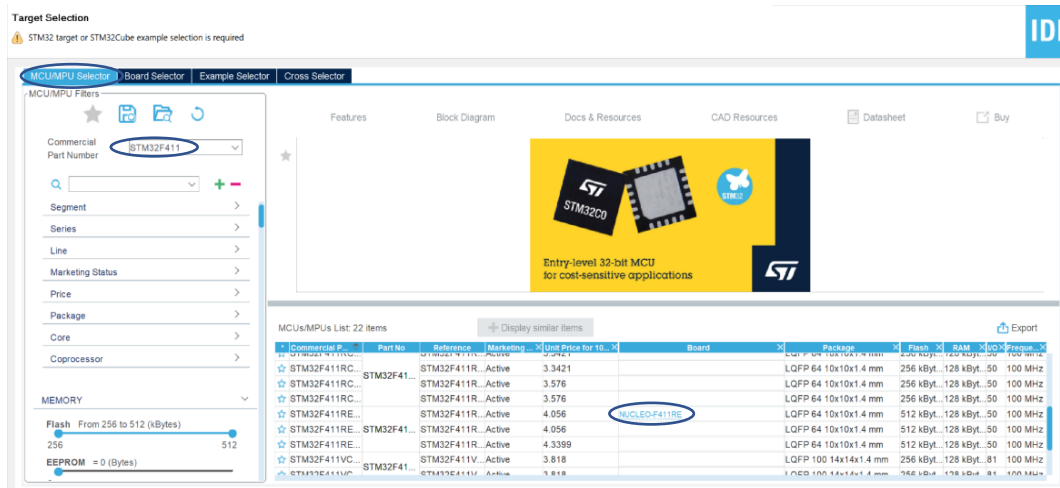
Programação BARE METAL e uso de estruturas em C

PARTE 1 – Codificação básica para escrita em uma porta (LED):

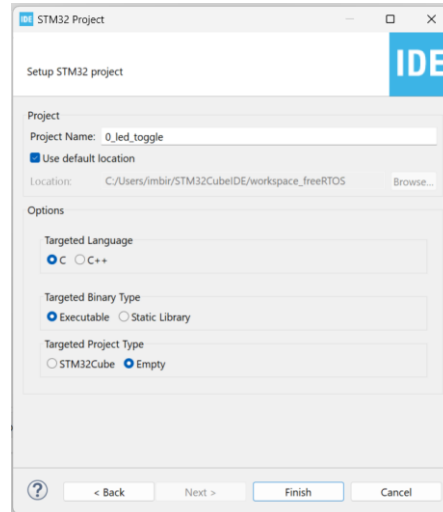
- 1- Abra o STM32CUBEIDE. Selecione a opção **File->New->STM32 Project**



- 2- Na opção **->MCU/MPU Selector** entre com a referência da CPU a ser usada ou o modelo da placa de desenvolvimento a ser utilizada em **->Board Selector**.



- 3- Click em **->next** e preencha os campos de setup do projeto conforme abaixo. Após preencher o **nome do projeto** e a **localização do ambiente de trabalho (workspace)** click em **->finish**.



- 4- Abra o arquivo main.c e delete todo o conteúdo do arquivo. Aqui vamos iniciar a programa sem considerar a inclusão de qualquer biblioteca já feita. Nós iremos desenvolver rotinas básicas em baixo nível para acessar recursos do microcontrolador. Nessa prática estaremos interessados em acessar o LED na placa de desenvolvimento NÚCLEO. Observe no manual da placa Núcleo onde está ligado o LED verde do usuário.

6.4 LEDs

The tricolor LED (green, orange, red) LD1 (COM) provides information about ST-L communication status. LD1 default color is red. LD1 turns to green to indicate that communication is in progress between the PC and the ST-LINK/V2-1, with the following setup:

- Slow blinking Red/Off: at power-on before USB initialization
- Fast blinking Red/Off: after the first correct communication between the PC and ST-LINK/V2-1 (enumeration)
- Red LED On: when the initialization between the PC and ST-LINK/V2-1 is complete
- Green LED On: after a successful target communication initialization
- Blinking Red/Green: during communication with the target
- Green On: communication finished and successful
- Orange On: Communication failure

User LD2: the green LED is a user LED connected to ARDUINO® signal D13 corresponding to STM32 I/O PA5 (pin 21) or PB13 (pin 34) depending on the STM32 target. Refer to Table 11 to Table 25 when:

- the I/O is HIGH value, the LED is on
- the I/O is LOW, the LED is off

LD3 PWR: the red LED indicates that the STM32 part is powered and +5V power is available.

- 5- Escreve o código a seguir no arquivo main.c e depois compile e carregue o executável na placa NÚCLEO. Observe o LED VERDE na placa NÚCLEO.



```
1 //onde o led está conectado?
2 //Port: A
3 //Pin: 5
4
5 #define PERIPH_BASE (0x40000000UL) // Ver em datasheet MEMORY MAP e
6 #define AHB1PERIPH_OFFSET (0x00020000UL) // BLOCK DIAGRAM em lista de figuras
7 #define AHB1PERIPH_BASE (PERIPH_BASE + AHB1PERIPH_OFFSET)
8 #define GPIOA_OFFSET (0x0000UL)
9
10 #define GPIOA_BASE (AHB1PERIPH_BASE + GPIOA_OFFSET)
11
12 #define RCC_OFFSET (0x3800UL)
13 #define RCC_BASE (AHB1PERIPH_BASE + RCC_OFFSET)
14
15
16 #define AHB1ENR_OFFSET (0x30UL) // endereço offset do registrador que habilita o clock do
17 // barramento de periféricos AHB1 - reference manual PAG. 117
18 // Endereço de memória do registrador (RCC_AHB1ENR)
19 #define RCC_AHB1ENR (*(volatile unsigned int *)(RCC_BASE + AHB1ENR_OFFSET))
20 // Definição da constante para habilitar o clock do barramento AHB1 para acessar GPIOA
21 #define GPIOAEN (1U<<0) // 0b 0000 0000 0000 0000 0000 0000 0001
22
23
24 #define MODE_R_OFFSET (0x00UL) // endereço Offset do registrador que define o modo de um
25 // determinado i/o - reference manual PAG. 157
26 // Endereço de memória do registrador (GPIOx_MODER)
27 #define GPIOA_MODE_R (*(volatile unsigned int *)(GPIOA_BASE + MODE_R_OFFSET))
28
29 #define OD_R_OFFSET (0x14UL) // endereço offset do registrador de saída de um i/o
30 // (OUTPUT DATA) - reference manual PAG. 159
31 // Endereço de memória do registrador de saída (GPIOx_ODR)
32 #define GPIOA_OD_R (*(volatile unsigned int *)(GPIOA_BASE + OD_R_OFFSET))
33
34
35 #define PIN5 (1U<<5) // macro para definir uma constante
36 #define LED_PIN PIN5
37
38 /*
39 * |= (1U<<10) // Set bit 10 para 1
40 * &= ~(1U<<11) // Set bit 11 para 0*/
41
42 int main(void)
43 {
44     /*1. Habilita o clock para acessar o GPIOA*/
45     RCC_AHB1ENR |= GPIOAEN;
46
47     /*2.Set PA5 como pino de saída*/
48     GPIOA_MODE_R |= (1U<<10); // Set bit 10 to 1
49     GPIOA_MODE_R &= ~(1U<<11); // Set bit 11 to 0
50
51     while(1)
52     {
53         /*3: faz o toggle em PA5*/
54         GPIOA_OD_R ^= LED_PIN;
55         for(int i=0;i<100000;i++){
56
57         }
58     }
59 }
```

PARTE 2 – Codificação usando estruturas de registradores:

Uma "struct" (abreviatura de "structure") é uma estrutura de dados que permite agrupar vários tipos de dados em um único objeto. É uma das principais estruturas de dados em C e é usada para representar objetos complexos, como **registros**, objetos ou entidades.

1. Digite o código abaixo utilizando as definições de estruturas (struct) básicas para registradores do RCC (clock) e GPIO.



```
main.c X
1 #include <stdint.h>
2
3 #define PERIPH_BASE      (0x40000000UL)
4 #define AHB1PERIPH_OFFSET (0x00020000UL)
5 #define AHB1PERIPH_BASE  (PERIPH_BASE + AHB1PERIPH_OFFSET)
6 #define GPIOA_OFFSET     (0x000000UL)
7
8 #define GPIOA_BASE       (AHB1PERIPH_BASE + GPIOA_OFFSET)
9
10 #define RCC_OFFSET       (0x3800UL)
11 #define RCC_BASE         (AHB1PERIPH_BASE + RCC_OFFSET)
12
13 #define GPIOAEN          (1U<<0) // 0b 0000 0000 0000 0000 0000 0000 0001
14
15 #define PIN5              (1U<<5)
16 #define LED_PIN          PIN5
17
18 typedef struct
19 {
20     volatile uint32_t DUMMY[12]; /* Atribui 12 x 4 = 48 endereços fictícios */
21     volatile uint32_t AHB1ENR; /* !< RCC AHB1 registrador do clock de periféricos, Endereço offset: 0x30 */
22 } RCC_TypeDef;
23
24
25 typedef struct
26 {
27     volatile uint32_t MODER; /* !< GPIO port mode register, Endereço offset: 0x00 */
28     volatile uint32_t DUMMY[4];
29     volatile uint32_t ODR; /* !< GPIO port output data register, Endereço offset: 0x14 */
30 } GPIO_TypeDef;
31
32
33 #define RCC ((RCC_TypeDef*) RCC_BASE) // Define endereço base da estrutura RCC_TypeDef como sendo RCC_BASE
34 #define GPIOA ((GPIO_TypeDef*) GPIOA_BASE) // Define endereço base da estrutura GPIO_TypeDef como sendo GPIOA_BASE
35
36 int main(void)
37 {
38     /*1. Habilita o clock para acessar GPIOA*/
39     RCC->AHB1ENR |= GPIOAEN;
40
41     /*2. Set PA5 como pino de saída*/
42     GPIOA->MODER |= (1U<<10); // Set bit 10 to 1
43     GPIOA->MODER &=~(1U<<11); // Set bit 11 to 0
44
45     while(1)
46     {
47         GPIOA->ODR ^= LED_PIN;
48         for(int i=0; i<1000000; i++){
49
50         }
51     }
```

2. Compile e carregue o código na placa NÚCLEO.

PARTE 3A – Codificação incluindo arquivos cabeçalho (header .h):

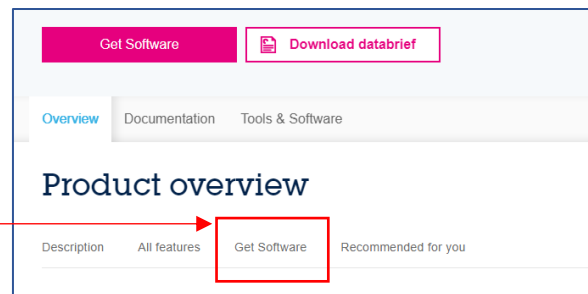
```
1 #include "stm32f4xx.h"
2 #define GPIOAEN          (1U<<0)
3 #define PIN5              (1U<<5)
4 #define LED_PIN          PIN5
5
6 int main(void)
7 {
8
9     RCC->AHB1ENR |= GPIOAEN;
10
11     GPIOA->MODER |= (1U<<10);
12     GPIOA->MODER &=~(1U<<11);
13
14     while(1)
15     {
16         GPIOA->ODR ^= LED_PIN;
17         for(int i=0; i<1000000; i++){
18
19         }
20 }
```

Arquivo header contendo as estruturas de registradores do processador



1. DOWNLOAD OS ARQUIVOS HEADER DO STM32F4
2. Entre no site da ST e em produtos procure pelo STM32F4
(https://www.st.com/content/st_com/en/search.html?q=STM32F4-t=tools-page=1)
3. Selecione o STM32CUBE4 conforme abaixo:

STSW-STM32066	ATIVO	Software embutido	Mcu mpu software embutido	Emulação de EEPROM em microcontroladores STM32F40x/STM32F41x (AN3969)
STM32CubeF4	ATIVO	Software embutido	Mcu mpu software embutido	Pacote STM32Cube MCU para a série STM32F4 (HAL, APIs de camada baixa e CMSIS, USB, TCP/IP, sistema de arquivos, RTOS, gráfico - e exemplos executados em placas ST)



4. Click STM32F4CUBE4 (Receber as últimas)

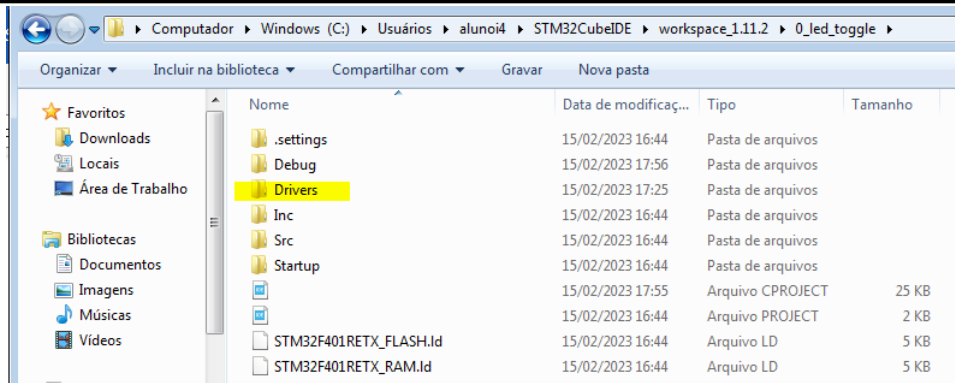
Obter software

Número da peça	Descrição geral	Download	Todas versões
+ Patch_CubeF4	Patch para STM32CubeF4	Receber as últimas	Selecione a versão
+ STM32CubeF4	Pacote MCU STM32Cube para a série STM32F4	Receber as últimas Obter do GitHub	Selecione a versão

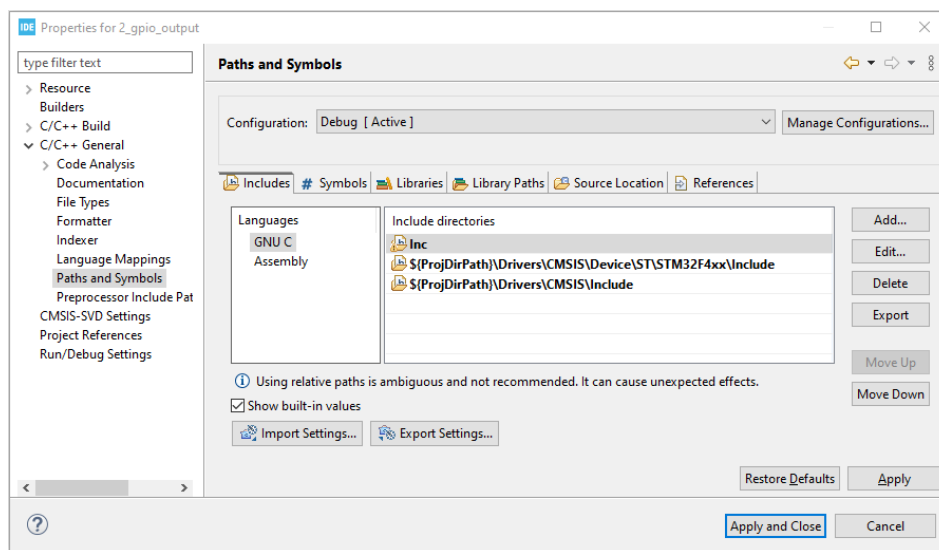
5. Aceite o contrato de licença;
6. Fazer o login na ST.

PARTE 3B – Incluindo arquivos cabeçalho (header .h) diretamente no workspace:

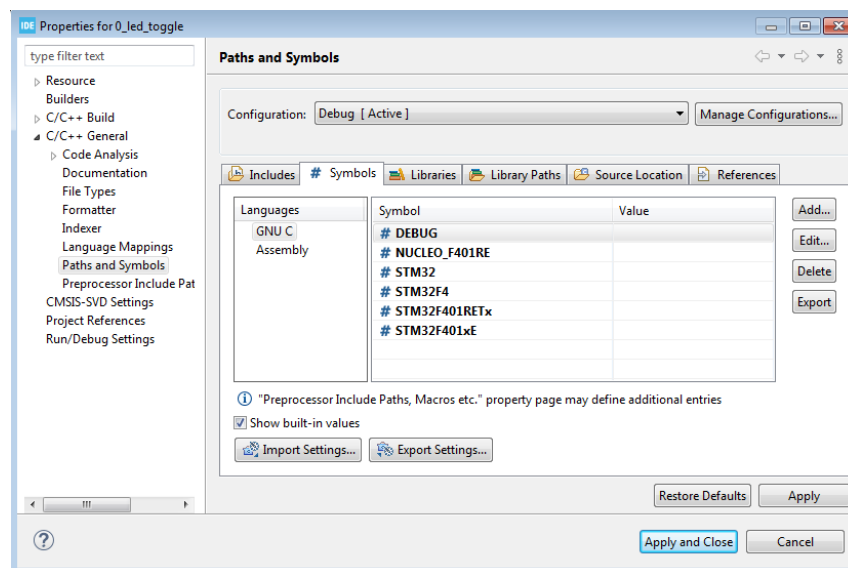
1. Baixar o arquivo Drivers.zip;
2. Descompactar e copiar a pasta Drivers para o workspace do projeto;



3. Abra a caixa de diálogo das propriedades do projeto. Na opção includes, insira o caminho das duas pastas **include** do diretório **Drivers**, tal como mostrado nas figuras abaixo:



4. Em **Symbols** adiciona a referência do microcontrolador a ser usado no projeto, (STM32F401xE ou STM32F411xE) dependendo da placa NÚCLEO utilizada, conforme a figura abaixo:





PARTE 4 – Escrevendo no registrador BSRR:

1. Altere a linha 16 do código escrevendo 1 no registrador BSRR correspondente à saída do pino 5 – LED.

```
main.c X
1 #include "stm32f4xx.h"
2
3 #define GPIOAEN      (1U<<0)
4 #define PIN5         (1U<<5)
5 #define LED_PIN      PIN5
6
7 int main(void)
8 {
9     RCC->AHB1ENR |=GPIOAEN;
10    GPIOA->MODER |= (1U<<10);
11    GPIOA->MODER &=~(1U<<11);
12
13    while(1)
14    {
15        // GPIO port bit set/reset register - offset 0x18 - RM 160
16        GPIOA->BSRR = LED_PIN;
17        for(int i=0;i<100000;i++){ }
18
19        GPIOA->BSRR = (1U<<21);
20        for(int i=0;i<100000;i++){ }
21
22    }
23 }
24
```